

Modelling Projectile Motion with Quadratic Drag: Ideal vs. Realistic Trajectories

A numerical comparison of drag-free and drag-affected cannonball trajectories using RK4 integration.

Abstract

Classical projectile motion, the parabola every physics student derives, assumes away air resistance entirely. This project models a cannonball's trajectory under **quadratic drag** (the physically correct regime for a fast-moving, human-scale projectile) and compares it against the idealized no-drag case. Because drag introduces a nonlinear coupling between the horizontal and vertical equations of motion, no closed-form solution exists; the drag case is solved with a fourth-order Runge-Kutta (RK4) integrator. The results show that drag doesn't just shrink the trajectory uniformly, it reshapes it, breaking the symmetric parabola into a skewed curve and shifting the optimal launch angle away from the textbook 45° .

1. Introduction

A projectile launched with the only force being gravity follows a symmetric parabolic arc, a result so clean it's often the first non-trivial physics derivation students see. But real projectiles move through air, and at the speeds involved (on the order of 100 m/s for an actual cannonball), the drag force is not negligible and does not scale linearly with velocity. It scales with the *square* of velocity, a regime known as **quadratic drag**, dominant whenever the Reynolds number is high (fast-moving, human-scale objects in air, as opposed to, say, dust settling in still air).

This project builds both models side by side:

1. The **ideal case** — ordinary projectile motion, solved analytically.
2. The **drag case** — the same problem with quadratic air resistance added, solved numerically.

and compares them on range, maximum height, flight time, and optimal launch angle.

2. Theoretical Background

2.1 The Ideal Case

With gravity as the only force, Newton's second law gives:

$$a_x = 0 \text{ and } a_y = -g$$

Integrating twice (with initial speed v_0 and launch angle θ gives the closed-form position equations:

$$x(t) = v_0 \cos \theta t,$$

$$y(t) = v_0 \sin \theta t - \frac{1}{2}gt^2$$

Setting $y(t) = 0$ and solving for the nonzero root gives the total flight time:

$$t_{flight} = \frac{2v_0 \sin \theta}{g}$$

from which range and maximum height follow directly:

$$Range = v_0 \cos \theta \times t_{flight} ,$$

$$Max Height = \frac{(v_0 \sin \theta)^2}{2g}$$

This is the entire ideal-case solution, a handful of algebraic formulas, no iteration required.

2.2 The Drag Case

Quadratic drag introduces a force opposing the velocity vector, with magnitude proportional to v^2 :

$$\vec{F}_d = -\frac{1}{2}\rho C_d A v \vec{v} ,$$

$$v = \sqrt{v_x^2 + v_y^2}$$

Where;

ρ is air density,

C_d the drag coefficient (≈ 0.47 for a sphere), and

A the cross-sectional area.

Newton's second law split's into:

$$a_x = -bv v_x ,$$

$$a_y = -g - bv v_y$$

Where;

$$b = \frac{\rho C_d A}{2m}$$

The key difference from the ideal case: $v = \sqrt{v_x^2 + v_y^2}$ couples the two equations together. Neither can be solved independently of the other, and no elementary closed-form solution exists for this coupled nonlinear system, it must be integrated numerically.

3. Numerical Method: Fourth Order Runge-Kutta Method (RK4)

Define the state vector:

$$\vec{y} = (x, y, v_x, v_y)$$

Then the system becomes:

$$\dot{x} = v_x ,$$

$$\dot{y} = v_y ,$$

$$a_x = -bv v_x,$$

$$a_y = -g - bv v_y$$

Derivative function:

$$f(\vec{y}) = (v_x, v_y, -bv v_x, -g - bv v_y)$$

A fourth-order Runge-Kutta (RK4) integrator advances this state forward in fixed timesteps Δt . Rather than using a single slope estimate per step (as the simpler, less accurate Euler's method does), RK4 samples the slope four times per step and combines them as a weighted average:

$$k_1 = f(\vec{y}_n), k_2 = f\left(\vec{y}_n + \frac{1}{2} \Delta t k_1\right), k_3 = f\left(\vec{y}_n + \frac{1}{2} \Delta t k_2\right), k_4 = f\left(\vec{y}_n + \Delta t k_3\right)$$

$$\vec{y}_{n+1} = \vec{y}_n + \frac{\Delta t}{6} (k_1 + 2k_2 + 2k_3 + k_4)$$

- k_1 is the slope at the start of the interval.
- k_2, k_3 are two successive refinements of the slope at the interval's midpoint.
- k_4 is the slope at the (estimated) end of the interval.

This gives local error of $O(\Delta t^5)$ per step, far better than Euler's of $O(\Delta t^2)$, at the cost of four function evaluations instead of one. Because the drag dynamics are nonlinear but smooth (no stiffness), a fixed step size of $\Delta t = 0.001$ s proved accurate and stable.

Landing detection: since the integrator only checks the sign of y after each fixed step, the exact moment of impact is refined via linear interpolation between the last two points straddling $y = 0$, rather than requiring an impractically small step size to land exactly on the ground.

4. Implementation

The full model was implemented in Python (NumPy for the numeric, Matplotlib for visualization). The code:

```
1. import numpy as np
2. import matplotlib.pyplot as plt
3.
4. # -----
5. # Physical parameters (edit these to model a different cannonball)
6. # -----
7. g = 9.81 # m/s^2
8. m = 5.0 # kg, projectile mass
9. diam = 0.10 # m, projectile diameter (a chunky 10 cm iron ball)
10. Cd = 0.47 # drag coefficient of a sphere
11. rho = 1.225 # kg/m^3, air density at sea level
12.
13. v0 = 120.0 # m/s, muzzle velocity
14. theta0 = 45.0 # degrees, launch angle
15.
16. A = np.pi * (diam / 2) ** 2 # cross-sectional area
17. b = 0.5 * rho * Cd * A / m # effective drag parameter (1/m)
18.
19. print(f"Drag parameter b = {b:.6f} 1/m")
20. print(f"(For reference, ballistic coefficient-like scale: 1/b = {1/b:.1f} m)")
21.
22. # -----
23. # Ideal (no drag) case: closed form
```

```

24. # -----
25. theta = np.radians(theta0)
26. vx0, vy0 = v0 * np.cos(theta), v0 * np.sin(theta)
27.
28. t_flight_ideal = 2 * vy0 / g
29. t_ideal = np.linspace(0, t_flight_ideal, 400)
30. x_ideal = vx0 * t_ideal
31. y_ideal = vy0 * t_ideal - 0.5 * g * t_ideal**2
32.
33. range_ideal = vx0 * t_flight_ideal
34. max_h_ideal = vy0**2 / (2 * g)
35.
36. # -----
37. # Drag case: RK4 numerical integration
38. # -----
39. def derivs(state):
40.     x, y, vx, vy = state
41.     v = np.hypot(vx, vy)
42.     ax = -b * v * vx
43.     ay = -g - b * v * vy
44.     return np.array([vx, vy, ax, ay])
45.
46. def simulate(v0, theta0, dt=0.001, t_max=100.0):
47.     theta = np.radians(theta0)
48.     state = np.array([0.0, 0.0, v0 * np.cos(theta), v0 * np.sin(theta)])
49.     traj = [state.copy()]
50.     t = 0.0
51.     while state[1] >= 0 and t < t_max:
52.         k1 = derivs(state)
53.         k2 = derivs(state + 0.5 * dt * k1)
54.         k3 = derivs(state + 0.5 * dt * k2)
55.         k4 = derivs(state + dt * k3)
56.         state = state + (dt / 6) * (k1 + 2 * k2 + 2 * k3 + k4)
57.         t += dt
58.         traj.append(state.copy())
59.     traj = np.array(traj)
60.     # linear interpolation to land exactly at y = 0
61.     x1, y1 = traj[-2, 0], traj[-2, 1]
62.     x2, y2 = traj[-1, 0], traj[-1, 1]
63.     frac = y1 / (y1 - y2)
64.     x_land = x1 + frac * (x2 - x1)
65.     t_land = (t - dt) + frac * dt
66.     return traj, x_land, t_land
67.
68. traj, range_drag, t_flight_drag = simulate(v0, theta0)
69. x_drag, y_drag = traj[:, 0], traj[:, 1]
70. max_h_drag = y_drag.max()
71.
72. # -----
73. # Finding optimal launch angle under drag (sweep) vs ideal (always 45)
74. # -----
75. angles = np.arange(20, 71, 1)
76. ranges_drag = []
77. for a in angles:
78.     _, r, _ = simulate(v0, a)
79.     ranges_drag.append(r)
80. best_angle_drag = angles[np.argmax(ranges_drag)]
81.
82. # -----
83. # Report
84. # -----
85. print("\n--- Comparison at theta0 = {:.0f} deg, v0 = {:.0f} m/s ---".format(theta0, v0))
86. print(f"{'Metric':<20}{'Ideal':>15}{'With drag':>15}")
87. print(f"{'Range (m)':<20}{range_ideal:>15.1f}{range_drag:>15.1f}")
88. print(f"{'Max height (m)':<20}{max_h_ideal:>15.1f}{max_h_drag:>15.1f}")
89. print(f"{'Flight time (s)':<20}{t_flight_ideal:>15.2f}{t_flight_drag:>15.2f}")
90. print(f"\nOptimal launch angle for max range under drag: {best_angle_drag} deg (vs 45 deg ideal)")
91.
92. # -----

```

```

93. # Plot
94. # -----
95. fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(13, 5))
96.
97. ax1.plot(x_ideal, y_ideal, label="No drag (ideal)", lw=2, color="#1f77b4")
98. ax1.plot(x_drag, y_drag, label="Quadratic drag", lw=2, color="#d62728")
99. ax1.set_xlabel("Horizontal distance (m)")
100. ax1.set_ylabel("Height (m)")
101. ax1.set_title(f"Trajectory comparison ($v_0$={v0} m/s, $\theta$={theta}\u00b0)")
102. ax1.legend()
103. ax1.grid(alpha=0.3)
104. ax1.set_ylim(bottom=0)
105.
106. ax2.plot(angles, ranges_drag, lw=2, color="#d62728", label="Range with drag")
107. ax2.axvline(45, color="#1f77b4", lw=2, linestyle="--", label="Ideal optimum (45\u00b0)")
108. ax2.axvline(best_angle_drag, color="#d62728", lw=1, linestyle=":",
109.             label=f"Drag optimum ({best_angle_drag}\u00b0)")
110. ax2.set_xlabel("Launch angle (deg)")
111. ax2.set_ylabel("Range (m)")
112. ax2.set_title("Range vs. launch angle")
113. ax2.legend()
114. ax2.grid(alpha=0.3)
115.
116. plt.tight_layout()
117. plt.show()
118.

```

The drag parameter b was computed from physical constants for a representative cannonball: a 5 kg iron sphere, 10 cm in diameter, at sea-level air density ($\rho = 1.225 \text{ kg/m}^3$, $C_d = 0.47$):

$$b = \frac{\rho C_d A}{2m} = \frac{1.225 \times 0.47 \times \pi(0.05)^2}{2 \times 5.0} \approx 0.000452 \text{ m}^{-1}$$

4.1 Refinement: Adaptive-Step Integration with `scipy.integrate.solve_ivp`

The hand-rolled RK4 above uses a fixed step size ($dt = 0.001 \text{ s}$) and locates the landing point after the fact via linear interpolation. A more robust implementation replaces both of these with tools purpose-built for the job: `scipy.integrate.solve_ivp`, using the adaptive-step RK45 method (Dormand-Prince), and a **terminal event function** that lets the solver find the exact landing time via root-finding rather than approximating it:

```

1. import numpy as np
2. import matplotlib.pyplot as plt
3. from scipy.integrate import solve_ivp
4.
5. g = 9.81                # Gravity (m/s^2)
6. rho = 1.225             # Air density at sea level (kg/m^3)
7. Cd = 0.47              # Drag coefficient for a sphere
8. r = 0.05               # Radius of the cannonball (m)
9. m = 5.0                # Mass of the cannonball (kg)
10. A = np.pi * (r**2)    # Cross-sectional area of the cannonball (m^2)
11. theta = 42.0           # Launch angle (degrees)
12.
13. v0 = 120.0             # Initial velocity (m/s)
14. vx0 = v0 * np.cos(np.radians(theta)) # Initial x-velocity
15. vy0 = v0 * np.sin(np.radians(theta)) # Initial y-velocity
16. x0, y0 = 0.0, 0.0     # Initial position (m)
17. b = 0.5 * rho * Cd * A / m # effective drag parameter (1/m)
18.
19. #-----
20. #the ideal case without drag for comparison
21. #-----

```

```

22. t_flight = 2 * vy0 / g
23. t = np.linspace(0, t_flight, 400) # 400 points from t=0 to landing
24. x = vx0 * t
25. y = vy0 * t - 0.5 * g * t**2
26.
27. #-----
28. #trajectory with quadratic drag
29. #-----
30. def derivatives(t, state):
31.     x, y, vx, vy = state
32.     v = np.hypot(vx, vy) # Total speed
33.     ax = -b * v * vx # Acceleration in x due to drag
34.     ay = -g - b * v * vy # Acceleration in y due to gravity and drag
35.     return [vx, vy, ax, ay]
36.
37. def hit_ground(t, state):
38.     return state[1] # Stop integration when y (height) is zero
39. hit_ground.terminal = True
40. hit_ground.direction = -1
41.
42. t_span = (0, 100) # Time span for the simulation
43. initial_state = [x0, y0, vx0, vy0] # Initial state vector
44. t_eval = np.linspace(0, 100, 1000) # Time points to evaluate the solution
45.
46. solution = solve_ivp(derivatives, t_span, initial_state, method='RK45', events=hit_ground,
t_eval=t_eval)
47.
48. # --- Plot the Trajectory ---
49. x_vals = solution.y[0]
50. y_vals = solution.y[1]
51.
52. # --- Print Results ---
53. print(f"Distance traveled: {x_vals[-1]:.2f} meters")
54. print(f"Flight time: {solution.t[-1]:.2f} seconds")
55.
56. plt.figure(figsize=(10, 5))
57. plt.plot(x_vals, y_vals, label=f'Quadratic Drag (v0 = {v0} m/s)', color='firebrick',
linewidth=2)
58. plt.plot(x, y, label=f'No Drag (v0 = {v0} m/s)', color='navy', linewidth=2, linestyle='--')
59. plt.title('Cannonball Trajectory with Quadratic Air Resistance')
60. plt.xlabel('Horizontal Distance (m)')
61. plt.ylabel('Vertical Height (m)')
62. plt.axhline(0, color='black', linewidth=1)
63. plt.ylim(bottom=0)
64. plt.grid(True, linestyle='--', alpha=0.7)
65. plt.legend()
66. plt.show()
67.

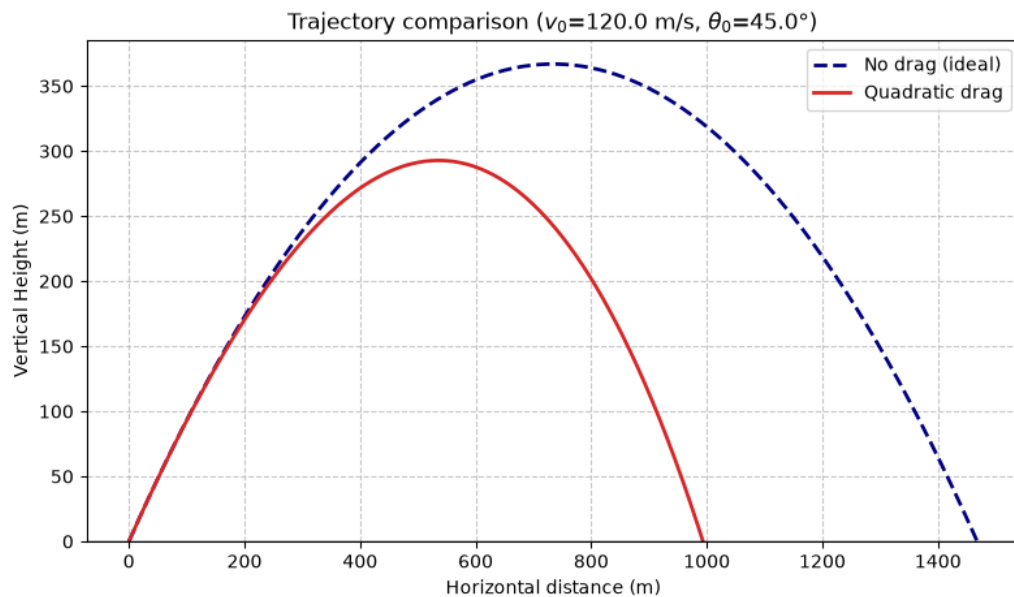
```

Two improvements over the fixed-step version: **adaptive step sizing** automatically takes finer steps where the trajectory curves quickly (e.g. near apex) and coarser steps where it doesn't, rather than using one fixed dt everywhere; and **event-based termination** finds the precise moment $y = 0$ is crossed through dense-output root-finding, rather than the linear-interpolation approximation the hand-rolled version relies on. At the drag-optimal launch angle found earlier (42°), this implementation gives a range of 996.19 m and a flight time of 14.61 sec, consistent with the fixed-step results, confirming both implementations converge to the same physical answer.

5. Results

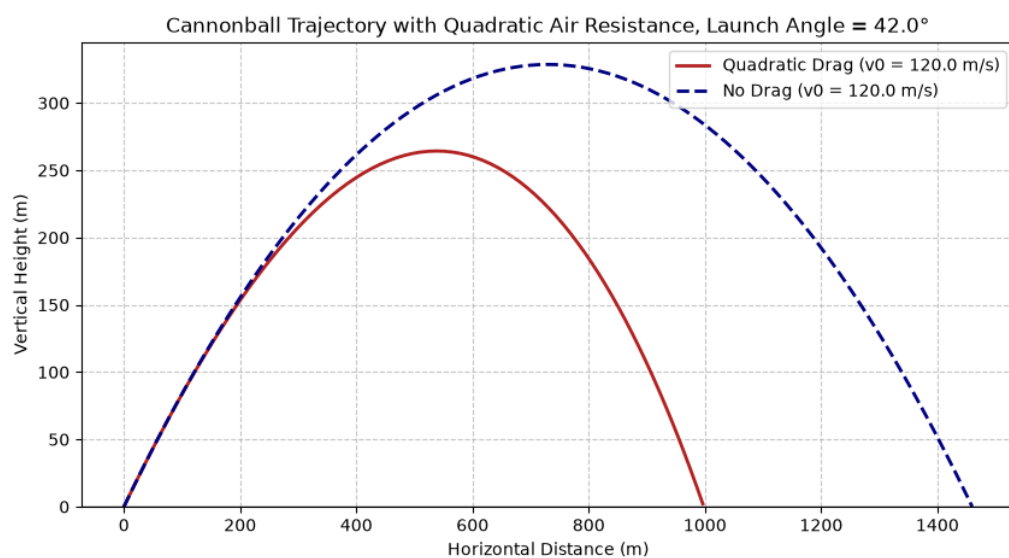
Simulating both models at $v_0 = 120$ m/s and $\theta_0 = 45^\circ$:

Metric	Ideal (no drag)	With drag
Range	1467.9 m	993.4 m
Max height	367.0 m	292.8 m
Flight time	17.30 s	15.42 s



At the optimum angle of $\theta_0 = 42^\circ$:

Metric	Ideal (no drag)	With drag
Range	1459.8 m	997.9 m
Max height	328.6 m	264.3 m
Flight time	16.37 s	14.65 s



Sweeping launch angle from 20° to 70° reveals that the drag case's optimal launch angle for maximum range is 42° , not the ideal case's 45° .

6. Discussion

Three findings stand out:

Range loss is substantial but not catastrophic. A 32% reduction in range for this ball's size and speed illustrates why real ballistics tables, for artillery, snipers, or any fast projectile, cannot use the ideal parabola formula and expect useful accuracy.

The trajectory shape is asymmetric, not just smaller. The drag trajectory rises nearly as steeply as the ideal one early on (drag hasn't yet had time to act at high initial speed) but falls more steeply than it rose. This happens because drag continuously bleeds horizontal velocity throughout the flight, while gravity's vertical pull continues unopposed by an equivalent decelerating force on the way down.

45° is no longer optimal. Under drag, a flatter launch angle (42° here) outperforms 45°, because it front-loads horizontal velocity before drag has as much time to sap it, a small but real correction that matters for anything from historical artillery gunnery to modern ballistic calculators.

A single parameter governs everything. The drag parameter $b = \frac{\rho C_d A}{2m}$, conceptually similar to the ballistic coefficient used in exterior ballistics, controls how far a given trajectory departs from the ideal case. Heavy, dense, streamlined projectiles (small A , large m) stay close to ideal; light or draggy ones diverge sharply.

7. Limitations and Future Work

This model holds air density and drag coefficient constant throughout the flight, which is a reasonable approximation for the altitudes involved here but breaks down for very high trajectories (density varies with altitude) or a wide range of Reynolds numbers (where C_d itself varies). Natural extensions include:

- **Altitude-dependent air density** via the barometric formula, for high-apogee trajectories.
- **Magnus effect** (spin-induced lift/curve), relevant for rifled projectiles.
- **Wind** as a background velocity field added to the relative velocity term.

8. Conclusion

Quadratic drag transforms projectile motion from a two-line algebraic result into a genuinely nonlinear system requiring numerical integration. Building both models side by side — one closed-form, one via RK4 — makes concrete something often left abstract in introductory physics: that the "ideal" parabola is a simplification whose accuracy depends entirely on how draggy the object is and how fast it's moving, and that even a well-understood numerical method like RK4 earns its complexity precisely where the physics stops being linear.